

SENSOR MONITORING SYSTEM

Embedded Linux Systems | Assignment 02

Report field	Value
Student	Nguyen Dang Quang
Student ID	SE201072
Target	ARMv7 QEMU Linux 5.15 Cortex-A9
Host	Ubuntu x86_64
Evidence	13 student-supplied terminal screenshots
Report date	10 October 2026
Status	Core + ARM RT validated; other rubric gaps disclosed

Technical report based on observed build outputs, screenshots, and test logs shared during development on 10 October 2026. This report distinguishes completed tests from features that have not yet been demonstrated.

Scope: Linux kernel sensor driver, POSIX multi-threaded application, IPC, memory management, debugging, profiling evidence, timing measurements, design review, and limitations.

1. Executive Summary

The Sensor Monitoring System (SMS) is an embedded Linux application that reads simulated temperature and humidity measurements from a custom character device. A four-thread user-space pipeline processes samples, raises alerts, and writes logs. ARM cross-compilation and integration on QEMU were successful.

The screenshots directly demonstrate a loaded kernel module, a live device node, active Sensor/Processor/Alert output, graceful shutdown, sample logging, and debugging and profiling artefacts. The host x86_64 version was used for Valgrind, GDB, strace and an initial log-observed jitter study followed by direct 2,000-sample wake-up latency measurements.

Evidence category	Observed outcome
ARM deployment	32-bit ARM EABI5 application, module and initramfs built
Integration	Sensor, Processor, Alert and Logger functioning
Memory test	Valgrind: 0 bytes remaining; 0 errors in 35 s host test
Target footprint	VmSize 34,540 KiB and reported VmRSS 4 KiB
Host timing	500.002 ms mean interval; 0.267 ms max observed jitter
Real-time follow-up	ARM SCHED_FIFO priority 50 confirmed; Host 2,000 wake-up samples

10 October 2026 update: follow-up tests confirmed ARM SCHED_FIFO priority 50 and mlockall; Host collected 1,000 direct wake-up samples per scheduling mode; perf recorded 8 SMS CPU-clock samples; GDB reproduced a temperature-spike alert using variable injection. Results on Host and ARM are reported separately. A demo video and certain other rubric items remain unverified.

2. System Architecture

The architecture separates data acquisition from processing, alert handling and storage. The driver creates `/dev/sms_sensor` and `/proc/sms_stats`. Sensor Thread acquires records and adds them to a 64-entry heap-backed circular buffer. Processor Thread consumes records, computes a rolling temperature average and emits typed alerts to a POSIX message queue.

Logical data flow

Kernel sensor driver -> Sensor Thread -> Circular Buffer (mutex + condition variable)

Circular Buffer -> Processor Thread -> POSIX alarm queue -> Alert Thread

Sensor/Processor/Alert -> Log Queue -> Logger Thread -> `/tmp/sensor.log`

Why separate threads?

A dedicated Sensor Thread helps isolate time-sensitive acquisition from the variable costs of alert display and filesystem I/O. Message queue delivery allows alerts to be produced independently of alert display. A Logger Thread decouples application activity from file writes.

Design traceability

The code and console screenshots demonstrate these components as a working system. The requirement document also requests a state-machine diagram, the resource-allocation/lock-order argument, and explicit trade-off discussion. Those topics are included below as design analysis, not as proof of formal correctness.

3. ARM Build and Deployable Artifacts

```
-rw-r--r-- 1 nguyendangquang nguyendangquang 12K Oct 10 13:38 sms_rss_app.log
-rw-r--r-- 1 nguyendangquang nguyendangquang 147 Oct 10 13:38 sms_rss_results.txt
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$ cd ~/embedded_asgn2

echo "===== ARM APPLICATION ====="
file app/sms_app_arm

echo
echo "===== ARM KERNEL DRIVER ====="
file driver/sms_sensor_driver.ko

echo
echo "===== INITRAMFS ====="
ls -lh output/initramfs_asgn2.cpio.gz
===== ARM APPLICATION =====
app/sms_app_arm: ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), statically linked, BuildID[sha1]=9229124251acec612a95012fd2f
===== ARM KERNEL DRIVER =====
driver/sms_sensor_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=f2a27525a7edde027a4023e3da7500a6c0d95
===== INITRAMFS =====
-rw-rw-r-- 1 nguyendangquang nguyendangquang 5.2M Oct 10 13:16 output/initramfs_asgn2.cpio.gz
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$
```

Figure 1. ARM ELF executable, ARM kernel module and compressed initramfs confirmed by the host terminal.

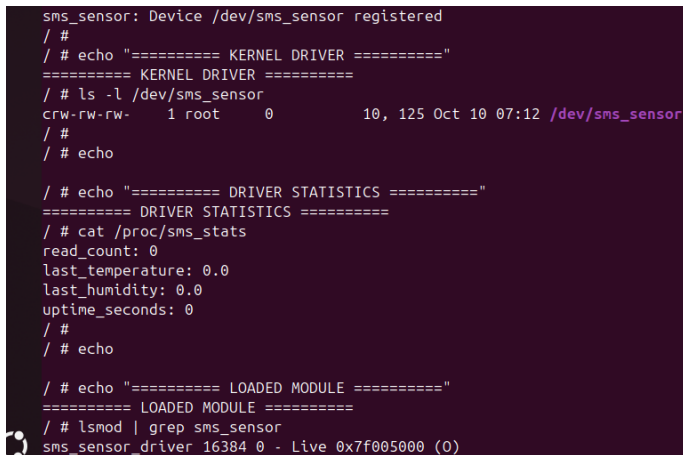
The executable was built as a statically linked ARM 32-bit EABI5 ELF file with debug information. The kernel module is an ARM relocatable ELF object. The initramfs archive is approximately 5.2 MB.

The use of a statically linked executable simplifies early initramfs deployment because the guest does not need a complete dynamic library set for sms_app. Nevertheless, memory accounting and executable size must not be confused: the on-disk file size is not identical to physical RSS.

Deployment command outline

QEMU machine: vexpress-a9; CPU: cortex-a9; RAM: 256 MB; kernel: zImage; device tree: vexpress-v2p-ca9.dtb; initramfs: initramfs_asgn2.cpio.gz; console: ttyAMA0; rdinit=/init.

4. Kernel Driver Verification

A screenshot of a terminal window with a dark purple background and light green text. The text shows the output of several commands in a shell. It starts with 'sms_sensor: Device /dev/sms_sensor registered', followed by a series of echo commands and a 'ls -l' command showing the file permissions and ownership of '/dev/sms_sensor'. Then, it shows the output of 'cat /proc/sms_stats' displaying statistics like 'read_count: 0', 'last_temperature: 0.0', 'last_humidity: 0.0', and 'uptime_seconds: 0'. Finally, it shows the output of 'lsmod | grep sms_sensor' displaying 'sms_sensor_driver 16384 0 - Live 0x7f005000 (0)'.

```
 sms_sensor: Device /dev/sms_sensor registered
 / #
 / # echo "===== KERNEL DRIVER ====="
 ===== KERNEL DRIVER =====
 / # ls -l /dev/sms_sensor
 crw-rw-rw-  1 root    0          10, 125 Oct 10 07:12 /dev/sms_sensor
 / #
 / # echo

 / # echo "===== DRIVER STATISTICS ====="
 ===== DRIVER STATISTICS =====
 / # cat /proc/sms_stats
 read_count: 0
 last_temperature: 0.0
 last_humidity: 0.0
 uptime_seconds: 0
 / #
 / # echo

 / # echo "===== LOADED MODULE ====="
 ===== LOADED MODULE =====
 / # lsmod | grep sms_sensor
 sms_sensor_driver 16384 0 - Live 0x7f005000 (0)
```

Figure 2. ARM guest confirms `/dev/sms_sensor`, `/proc/sms_stats` and loaded `sms_sensor_driver`.

The guest terminal shows the character device `/dev/sms_sensor` with major/minor 10,125. The procfs statistics entry reports `read_count`, `last_temperature`, `last_humidity` and `uptime_seconds`. At the instant captured, `read_count` is zero because the driver had just been loaded.

The driver interface also provides the `SMS_SET_INTERVAL` ioctl in the ARM integration build. A successful module load demonstrates registration, but the screenshot does not by itself prove all boundary conditions, error paths or an `insmod/rmmod` stress test.

5. Live Multi-threaded Processing

```
[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 39223 ms
[ALERT] Value: 9.1 | Threshold: 5.0
[SENSOR] Time=40223 ms | Temp=36.9 C | Hum=78.3%
[PROCESSOR] Sample=12 | Temp=36.9 C | Hum=78.3% | Avg(10)=26.78 C

[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 40223 ms
[ALERT] Value: 8.1 | Threshold: 5.0
[LOGGER] Periodic flush completed
[SENSOR] Time=41214 ms | Temp=20.8 C | Hum=51.2%
[PROCESSOR] Sample=13 | Temp=20.8 C | Hum=51.2% | Avg(10)=26.43 C
[SENSOR] Time=41720 ms | Temp=37.2 C | Hum=75.3%
[PROCESSOR] Sample=14 | Temp=37.2 C | Hum=75.3% | Avg(10)=26.33 C

[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 41720 ms
[ALERT] Value: 16.4 | Threshold: 5.0
[SENSOR] Time=42720 ms | Temp=20.8 C | Hum=38.6%
[PROCESSOR] Sample=15 | Temp=20.8 C | Hum=38.6% | Avg(10)=26.11 C
[SENSOR] Time=43721 ms | Temp=18.5 C | Hum=85.5%
[PROCESSOR] Sample=16 | Temp=18.5 C | Hum=85.5% | Avg(10)=24.64 C
```

Figure 3. Running ARM QEMU application with sensor samples, rolling averages and temperature-spike alerts.

The live console shows Sensor Thread samples, Processor Thread sample counts and Avg(10) temperature output, as well as temperature-spike alert messages. For example, an alert indicates a measured temperature change greater than the 5.0 C spike threshold.

The data path uses a bounded producer-consumer buffer. The processor works on samples independently of sensor acquisition, and the Alert Thread consumes messages from a POSIX queue. The expected sampling interval is 500 ms.

This screenshot demonstrates functional concurrency but does not establish freedom from all races or deadlocks. A separate lock-order argument and instrumentation would strengthen that claim.

6. Graceful Shutdown and Resource Cleanup

```
[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 41720 ms
[ALERT] Value: 16.4 | Threshold: 5.0
[SENSOR] Time=42720 ms | Temp=20.8 C | Hum=38.6%
[PROCESSOR] Sample=15 | Temp=20.8 C | Hum=38.6% | Avg(10)=26.11 C
[SENSOR] Time=43721 ms | Temp=18.5 C | Hum=85.5%
[PROCESSOR] Sample=16 | Temp=18.5 C | Hum=85.5% | Avg(10)=24.64 C

[ALERT] HIGH HUMIDITY
[ALERT] Timestamp: 43721 ms
[ALERT] Value: 85.5 | Threshold: 80.0
^C
[MAIN] Shutdown requested
[SENSOR] Thread stopped
[MAIN] Sensor Thread joined
[PROCESSOR] Thread stopped. Total processed: 16
[MAIN] Processor Thread joined
[ALERT] Thread stopped. Total alerts: 11
[MAIN] Alert Thread joined
[LOGGER] Thread stopped. Total written: 45
[MAIN] Logger Thread joined
[MAIN] Resources cleaned up
[MAIN] Application stopped
```

Figure 4. Ctrl+C followed by thread stop/join messages and resource cleanup.

After SIGINT, the application reports Shutdown requested, then stops and joins Sensor, Processor, Alert and Logger Threads. In the captured run, Processor processed 16 samples, Alert handled 11 events, and Logger wrote 45 records.

The teardown sequence drains or stops queues as appropriate, joins worker threads, removes the POSIX message queue and destroys synchronization resources. This trace is evidence of successful shutdown for the observed test.

Proposed application state transitions

INIT -> RUNNING after buffers, queues, device and worker threads initialize. RUNNING -> SHUTDOWN on SIGINT or fatal error. SHUTDOWN -> TERMINATED after joins, log flush and resource release. Initialization errors should also enter cleanup and return a nonzero exit status.

7. Logger and Traceable Evidence

```
[PROCESSOR] Time=31722 ms | Temp=24.3 C | Hum=80.3%
[ALERT] HIGH HUMIDITY | Time=31722 ms | Value=80.3 | Threshold=80.0
[ALERT] TEMPERATURE SPIKE | Time=31722 ms | Value=7.9 | Threshold=5.0
[SENSOR] Time=31722 ms | Temp=24.3 C | Hum=80.3%
[PROCESSOR] Time=32723 ms | Temp=38.2 C | Hum=88.4%
[ALERT] HIGH HUMIDITY | Time=32723 ms | Value=88.4 | Threshold=80.0
/ #
/ # echo

/ # echo "===== LOG STATISTICS ====="
===== LOG STATISTICS =====
/ # echo "Sensor:"
Sensor:
/ # grep -c '\[SENSOR\]' /tmp/sensor.log
16
/ #
/ # echo "Processor:"
Processor:
/ # grep -c '\[PROCESSOR\]' /tmp/sensor.log
16
/ #
/ # echo "Alert:"
Alert:
/ # grep -c '\[ALERT\]' /tmp/sensor.log
11
```

Figure 5. Saved application log contains Sensor, Processor and Alert entries with counts 16, 16 and 11.

The logged event total for the pictured run is 16 Sensor + 16 Processor + 11 Alert + two lifecycle records = 45 records. This agrees with the shutdown report from the same run. The logger uses a dedicated bounded queue and was separately unit-tested for periodic flush and file rotation.

The current deployed log path is /tmp/sensor.log. The assignment example names /tmp/sms.log; this is a naming mismatch that should be reconciled before submission. The assignment also requests complete timestamps and flush approximately every five seconds rather than flushing per line.

8. ARM Memory Footprint (RSS and VmSize)

```
nguyendangquang@nguyendangquang-Bellator:~$ cd ~/embedded_asgn2

echo "===== RSS MEMORY ANALYSIS ====="
cat logs/qemu_evidence/sms_rss_results.txt

echo
echo "===== MEMORY STATUS ====="
sed -n '/----- STATUS -----/,/----- STATM -----/p' \
  logs/qemu_evidence/sms_memory_verification.txt
===== RSS MEMORY ANALYSIS =====
Sensor Monitoring System - RSS Test
PID: 112
Time(s) | VmRSS(kB) | VmSize(kB)
5 | 4 | 34540
30 | 4 | 34540
55 | 4 | 34540
Application exit code: 0

===== MEMORY STATUS =====
----- STATUS -----
Name: sms_app
State: S (sleeping)
VmPeak: 35564 kB
VmSize: 34540 kB
VmRSS: 4 kB
VmData: 33052 kB
VmStk: 132 kB
VmExe: 424 kB
VmLib: 8 kB
Threads: 5
----- STATM -----
```

Figure 6. QEMU memory results at 5, 30 and 55 seconds and process status showing five threads.

Elapsed time	VmRSS (KiB)	VmSize (KiB)
5 s	4	34,540
30 s	4	34,540
55 s	4	34,540

The procfs measurements are stable at all three checkpoints and the program exited with code zero. However, the reported VmRSS value of only 4 KiB is unusually low for this workload; it is therefore reported as a kernel-provided figure, not a reliable statement about the total physical footprint.

In the same saved process status, VmData is 33,052 KiB and Threads is 5. The virtual footprint is consistent with allocation of separate thread stacks. An unchanged RSS value over three samples does not prove the absence of all leaks.

9. Virtual Memory Map Analysis

```
VMUCLD: 0 KB
Threads: 5
----- STATM -----
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$ cd ~/embedded_asgn2

echo "===== MEMORY MAPPING ====="

sed -n '/----- MEMORY MAPPINGS -----/, $p' \
  logs/qemu_evidence/sms_memory_verification.txt
===== MEMORY MAPPING =====
----- MEMORY MAPPINGS -----
00010000-0007a000 r-xp 00000000 00:02 125      /usr/bin/sms_app
0007a000-0007d000 r--p 00069000 00:02 125      /usr/bin/sms_app
0007d000-0007f000 rw-p 0006c000 00:02 125      /usr/bin/sms_app
0007f000-000a3000 rw-p 00000000 00:00 0        [heap]
74e00000-74e21000 rw-p 00000000 00:00 0
74e21000-74f00000 ---p 00000000 00:00 0
74f52000-74f53000 ---p 00000000 00:00 0
74f53000-75753000 rw-p 00000000 00:00 0
75753000-75754000 ---p 00000000 00:00 0
75754000-75f54000 rw-p 00000000 00:00 0
75f54000-75f55000 ---p 00000000 00:00 0
75f55000-76755000 rw-p 00000000 00:00 0
76755000-76756000 ---p 00000000 00:00 0
76756000-76f56000 rw-p 00000000 00:00 0
7ea61000-7ea82000 rw-p 00000000 00:00 0        [stack]
7ec64000-7ec65000 r-xp 00000000 00:00 0        [sigpage]
7ec65000-7ec66000 r--p 00000000 00:00 0        [vvar]
7ec66000-7ec67000 r-xp 00000000 00:00 0        [vdso]
ffff0000-ffff1000 r-xp 00000000 00:00 0        [vectors]
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$
```

Figure 7. ARM process mapping: executable mappings, heap, anonymous regions, stack and kernel helper pages.

Five interpretable region classes are visible: executable text (r-xp), read-only program data (r--p), writable data (rw-p), heap ([heap]), and main stack ([stack]). Further anonymous writable regions paired with no-access guard pages are consistent with POSIX thread stacks. [vdso], [vvar] and [vectors] are kernel-provided mappings.

The binary is statically linked, therefore shared-library mappings cannot be assumed from this ARM output. The memory map is evidence for region analysis, but it does not uniquely identify which anonymous mapping belongs to each named worker thread.

10. Valgrind Heap Analysis - Host x86_64

```
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$ cd ~/embedded_asgn2
echo "===== VALGRIND FULL PIPELINE ====="
cat logs/valgrind_full_pipeline.txt

echo
echo "===== THREAD RESULTS ====="
grep -E 'Total processed|Total alerts|Total written|Application stopped' \
  logs/valgrind_full_pipeline_output.txt
===== VALGRIND FULL PIPELINE =====
==8414== Memcheck, a memory error detector
==8414== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==8414== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==8414== Command: ./sms_app_host
==8414== Parent PID: 7878
==8414==
==8414==
==8414== HEAP SUMMARY:
==8414==   in use at exit: 0 bytes in 0 blocks
==8414==   total heap usage: 8 allocs, 8 frees, 10,776 bytes allocated
==8414==
==8414== All heap blocks were freed -- no leaks are possible
==8414==
==8414== For lists of detected and suppressed errors, rerun with: -s
==8414== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

===== THREAD RESULTS =====
[PROCESSOR] Thread stopped. Total processed: 70
[ALERT] Thread stopped. Total alerts: 140
[LOGGER] Thread stopped. Total written: 282
[MAIN] Application stopped
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$
```

Figure 8. Valgrind full-pipeline run on simulated Host sensor input: no reported leaks or memory errors.

Check	Observed result
Duration	35 seconds
Heap at exit	0 bytes in 0 blocks
Allocations / frees	8 / 8
Memory errors	0 errors from 0 contexts
Pipeline counters	70 processed; 140 alerts; 282 log messages

The host build uses a regular CSV file in place of the ARM kernel device, and the host-specific test bypasses the driver ioctl. The workload exercises temperature and humidity thresholds. Thus the clean Valgrind finding applies to this execution path only; it cannot replace ARM-kernel or every error-path analysis.

11. GDB Debugging Evidence

```
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at main.c:55
55 {
Num   Type           Disp Enb Address            What
1     breakpoint     keep y   0x00005555555556709 in main at main.c:55
      breakpoint already hit 1 time
2     breakpoint     keep y   0x000055555555575ed in sensor_thread_main at sensor_thread.c:85
3     breakpoint     keep y   0x00005555555557d97 in processor_thread_main at processor_thread.c:152
Id    Target Id      Frame
* 1   Thread 0x7ffff7fa7740 (LWP 8506) "sms_app_host" main () at main.c:55
#0   main () at main.c:55
[New Thread 0x7ffff7bfb6c0 (LWP 8509)]
[New Thread 0x7ffff73fe6c0 (LWP 8510)]
[New Thread 0x7ffff6bfd6c0 (LWP 8511)]
[Switching to Thread 0x7ffff6bfd6c0 (LWP 8511)]

Thread 4 "sms_app_host" hit Breakpoint 3, processor_thread_main (arg=0x7ffffff5a80) at processor_thread.c:152
152 {
Id    Target Id      Frame
1     Thread 0x7ffff7fa7740 (LWP 8506) "sms_app_host" clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:62
2     Thread 0x7ffff7bfb6c0 (LWP 8509) "sms_app_host" 0x00007ffff7c98e51 in __futex_abstimed_wait_common64 (private=21845, cancel=true, abs
3     Thread 0x7ffff73fe6c0 (LWP 8510) "sms_app_host" clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:62
* 4     Thread 0x7ffff6bfd6c0 (LWP 8511) "sms_app_host" processor_thread_main (arg=0x7ffffff5a80) at processor_thread.c:152
#0   processor_thread_main (arg=0x7ffffff5a80) at processor_thread.c:152
#1   0x00007ffff7c9cb84 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#2   0x00007ffff7d29ecc in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78
A debugging session is active.

        Inferior 1 [process 8506] will be killed.
```

Figure 9. GDB confirms breakpoints at main and processor_thread_main, live thread listing and a backtrace.

The debugger stopped at main.c:55, followed by a breakpoint in processor_thread_main at processor_thread.c:152. The thread listing and backtrace show pthread-created workers and their call stacks. The screenshot supports debugger integration and thread-state inspection.

Subsequent Host GDB tests reached check_sensor_alerts(), showed active threads and a backtrace, and injected previous_temp=25.0 C while current temperature was 42.5 C. The application emitted TEMPERATURE SPIKE, difference 17.5 C against threshold 5.0 C. This is debugger-injected state, not two distinct physical sensor samples. The exact assignment wording for a 45.0 C injection or a buffer-race scenario is not fully demonstrated.

12. System-call Profiling with strace

```
Inferior 1 [process 8506] will be killed.
Quit anyway? (y or n) [answered Y; input not from terminal]
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$ cd ~/embedded_asgn2

echo "===== STRACE SYSTEM CALLS ====="

for syscall in clone3 futex mq_timedreceive clock_nanosleep read write; do
    count=$(grep -Ec " ${syscall}\(" logs/strace_report.txt)
    echo "$syscall: $count"
done

echo
echo "===== CLEAN EXIT ====="
tail -8 logs/strace_report.txt
===== STRACE SYSTEM CALLS =====
clone3: 4
futex: 11661
mq_timedreceive: 3653
clock_nanosleep: 3189
read: 914
write: 3721

===== CLEAN EXIT =====
8518 13:59:56.428686 exit(0) = ?
8517 13:59:56.428728 <... futex resumed>) = 0
8518 13:59:56.428734 +++ exited with 0 +++
8517 13:59:56.428746 close(3) = 0
8517 13:59:56.428775 mq_unlink("sms_alert_queue") = 0
8517 13:59:56.428855 write(1, "LERT] HIGH HUMIDITY\n[ALERT] Time"... , 1912) = 1912
8517 13:59:56.428916 exit_group(0) = ?
8517 13:59:56.429336 +++ exited with 0 +++
```

Figure 10. strace system-call counts and successful process exit.

System call	Count	Interpretation
futex	11,661	Mutex/condition-variable waiting and signaling
write	3,721	Console output and log file writes
mq_timedreceive	3,653	Timed alert queue reception
clock_nanosleep	3,189	Periodic sensor and main-loop timing
read	914	Sensor file reads
clone3	4	Four POSIX worker threads

The trace ends with `mq_unlink` and `exit_group(0)`, consistent with orderly process termination. The saved trace used `strace -f -tt` rather than the rubric-required `strace -c` summary; therefore these are invocation counts from a text trace and not a `-c` time-percentage report.

13. Sampling Period and Measured Jitter

```
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$ cd ~/embedded_asgn2

echo "===== JITTER ANALYSIS ====="
cat logs/jitter_report.txt
===== JITTER ANALYSIS =====
SENSOR MONITORING SYSTEM - JITTER TEST
Environment: Ubuntu Host x86_64
Expected sampling interval: 500 ms
Number of samples: 61
Average interval: 500.002 ms
Minimum interval: 499.733 ms
Maximum interval: 500.195 ms
Average absolute jitter: 0.086 ms
Maximum absolute jitter: 0.267 ms
Application exit code: 0
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$
```

Figure 11. Host application-level sample log timing for 61 observations.

Metric	Observed result
Expected interval	500 ms
Number of samples	61
Average period	500.002 ms
Minimum / maximum period	499.733 / 500.195 ms
Average absolute jitter	0.086 ms
Maximum absolute jitter	0.267 ms

This original 61-observation figure used Host log-reception timestamps and is retained as historical baseline only. Later instrumented Host Sensor Thread tests measured `CLOCK_MONOTONIC` wake-up latency directly, with 1,000 readings for `SCHED_OTHER` and 1,000 for `SCHED_FIFO`. Complete Min/Avg/Max/P99 and histogram results appear in Section 20. Do not confuse observer-estimated interval error with direct wake-up latency.

14. Scheduling Policy and Real-Time Gap

```
nguyendangquang@nguyendangquang-Bellator: ~/embedded_asgn2$ cd ~/embedded_asgn2
echo "===== SCHEDULING REPORT ====="
cat logs/scheduling_report.txt
===== SCHEDULING REPORT =====
SENSOR MONITORING SYSTEM - SCHEDULING TEST
PID: 8797

  PID   TID  CLS RTPRIO PRI  NI COMMAND
  ----  ---  ---  -  ---  --  -
  8797   8797 TS    -   19   0 sms_app_host
  8797   8799 TS    -   19   0 sms_app_host
  8797   8800 TS    -   19   0 sms_app_host
  8797   8801 TS    -   19   0 sms_app_host
  8797   8802 TS    -   19   0 sms_app_host

===== CHRT DETAILS =====
--- Thread 8797 ---
pid 8797's current scheduling policy: SCHED_OTHER
pid 8797's current scheduling priority: 0
--- Thread 8799 ---
pid 8799's current scheduling policy: SCHED_OTHER
pid 8799's current scheduling priority: 0
--- Thread 8800 ---
pid 8800's current scheduling policy: SCHED_OTHER
pid 8800's current scheduling priority: 0
--- Thread 8801 ---
pid 8801's current scheduling policy: SCHED_OTHER
pid 8801's current scheduling priority: 0
--- Thread 8802 ---
pid 8802's current scheduling policy: SCHED_OTHER
pid 8802's current scheduling priority: 0
nguyendangquang@nguyendangquang-Bellator: ~/embedded_asgn2$
```

Figure 12. All five Host threads were inspected under `SCHED_OTHER` with real-time priority 0.

Five threads were visible: the main thread plus four workers. The tested scheduling policy was `SCHED_OTHER`, and every thread had scheduling priority zero. This is clear evidence of the actual baseline policy, not evidence of configured real-time priority.

This is the baseline policy from the earlier screenshot, before the Real-Time enhancement. On 10 October 2026 a separate ARM application was compiled and tested in QEMU: [RT] `mlockall` enabled and [RT] Requested=`SCHED_FIFO` Actual=`SCHED_FIFO` Priority=50. See Section 22. Host policy comparison and wake-up statistics appear in Section 20.

15. Synchronization and Deadlock Considerations

Shared resources and intended protection

Resource	Synchronization mechanism
Sensor circular buffer	pthread mutex + condition variable
Log Queue	pthread mutex + condition variable
Inter-thread alert delivery	POSIX message queue
Stop flags	C atomic_bool / signal-safe sig_atomic_t for handler
Log file	Written by Logger Thread

Deadlock prevention argument

The architecture avoids having the Sensor Thread write the log file itself and separates data-buffer access from alert-queue handling. A well-formed implementation should release a buffer mutex before blocking on a message queue and never hold a mutex while waiting on another resource that can require the first mutex. Shutdown should wake blocked consumers before pthread_join. These are design conditions to inspect against source, not a formal proof derived solely from console screenshots.

Potential race to review

A shutdown race can occur if a producer is still publishing log events after Logger Thread has stopped accepting messages. Ordering the joins and queue shutdown is essential. The observed normal teardown completes, but repeated stress testing would provide stronger evidence.

16. Rubric Compliance and Unfinished Work

Required item	Current evidence
Device/module and 4 threads	Demonstrated with guest screenshots
SIGINT shutdown	Demonstrated
Valgrind ≥ 30 s, no definite leak	Demonstrated on Host
QEMU RSS and process maps	Captured
GDB requested scenario	Alert-branch injection verified on Host; exact 45 C scenario still not shown
strace -c, 30 seconds, top 5 by time	Not demonstrated (trace counts only)
perf record/report hotspot	Recorded 8 genuine SMS samples on Host; limited confidence
SCHED_FIFO priority and mlockall	Verified on ARM QEMU, sensor FIFO 50; Host tested too
≥ 1000 jitter, P99, histogram, FIFO comparison	Verified on Host: 1000 each mode; not measured 1000 each on ARM
Command-line -i/-t/-h/-d/-v	Not demonstrated in current evidence
Humidity rolling average	Not demonstrated in shown console output
3-5 minute video	Not supplied
15-30 page technical PDF	Updated illustrated PDF within the 15-30 page range

This checklist is not a grade prediction. It highlights gaps between the source assignment instructions and the current evidence. In particular, the missing demo video, required strace -c evidence and ARM-scale timing comparison remain separately assessable gaps; the Host real-time measurements and verified ARM FIFO 50 are documented in Sections 20 and 22.

17. Engineering Reflection

Most challenging integration issues

The development sequence exposed several genuine embedded-Linux challenges: building an ARM-compatible driver and application, packaging the initramfs correctly, providing the QEMU device-tree file, correctly shutting down four interacting threads, and preserving evidence across QEMU guest restarts. The practical mitigation was to verify one interface at a time, capture terminal logs and move results to the Host before destroying a guest session.

What I would redesign

A future revision should unify configuration handling through command-line parsing; report both temperature and humidity rolling averages; make the host simulated sensor a first-class test backend; add explicit per-thread lifecycle and per-alert-type counters; and expose a structured interval/jitter report instead of relying on terminal output timestamps.

Future improvements

SCHED_FIFO priority 50 and mlockall have since been verified on ARM QEMU, and 1,000 wake-up latency samples per scheduler mode have been collected on the Host. Next priorities are a graceful SCHED_OTHER fallback when privilege checks fail, comparable 1,000-sample measurements inside ARM QEMU, more substantial perf sampling, the prescribed strace -c time-ranked report, additional fault-injection tests, and a reproducible Makefile and README. Persistent storage and remote telemetry could improve deployment usefulness.

This reflection is a proposed technical interpretation of the development work. It should be edited into the student's own words before academic submission.

18. Evidence Index and Reproducibility Notes

ARM binary and initramfs

Screenshot 01 (build), application ELF and module checked with file(1).

Kernel driver

Screenshot 02 (device node, proc stats, lsmod).

Application pipeline

Screenshots 03-05 (live operation, SIGINT teardown, saved logs).

Memory evidence

Screenshots 06-07 (RSS checkpoints, /proc/PID/maps).

Host diagnostics

Screenshots 08-10 (Valgrind, GDB and strace).

Timing and policy

Screenshots 11-12 (61-sample jitter and SCHED_OTHER policy).

Screenshot storage

Screenshot 13 shows local screenshots/ empty; files were shared in the conversation, not copied to that local folder.

Build reproducibility

A future submission package should include the driver Makefile, an application Makefile with ARM and x86 targets, kernel zImage and DTB location or a documented external prerequisite, and the exact boot and test commands. The student should verify that these artifacts match the expected naming rules.

19. Conclusion and References

The captured ARM QEMU evidence supports successful integration of the custom driver and a four-thread application. The application reads simulated sensor data, processes samples, publishes alerts, logs activity and exits cleanly after SIGINT. In the tested x86_64 Host pipeline, Valgrind reported all heap allocations released and zero memory errors. Timing, GDB, strace and scheduling output provide useful diagnostic evidence, with the measurement limitations explained in this report.

The follow-up verification established ARM SCHED_FIFO 50 with mlockall, a 2,000-sample direct Host jitter comparison, GDB alert-branch injection, and a small genuine perf hotspot sample. Remaining gaps include the exact instructor GDB scenario if strictly interpreted, strace -c time-ranked output, CLI -i/-t/-h/-d/-v evidence, humidity rolling average evidence, an ARM 1,000-sample comparison, and the 3-5 minute video.

References

ASSIGNMENT-02 | Embedded Linux Systems | Write Embedded Application - Sensor Monitoring System, instructor-provided document.

Linux manual pages: pthread_create(3), pthread_cond_wait(3), mq_open(3), clock_nanosleep(2), sched_setscheduler(2).

Student-provided ARM QEMU, Valgrind, GDB, strace, jitter and scheduling screenshots, 10 October 2026.

Valgrind 3.22.0 Memcheck output, test session on Ubuntu x86_64.

Academic integrity note

The course instructions require students to understand and author submitted code. This report is a draft for student review and should be checked against the student's implementation, supplemented with original reflections and adjusted to the instructor's submission requirements.

20. Real-Time Wake-up Latency: Follow-up Measurements

Test date: 10 October 2026. Environment: Ubuntu x86_64 Host, synthetic CSV input; Sensor Thread measures actual CLOCK_MONOTONIC wake-up time after absolute clock_nanosleep(). Interval: 500 ms. Exactly 1,000 readings were collected for each policy. These are wake-up lateness measurements in microseconds, not sampling-period drift.

Metric	SCHED_OTHER (priority 0)	SCHED_FIFO (priority 50)
Samples	1,000	1,000
Minimum	55 us	6 us
Average	80.02 us	17.52 us
Maximum	278 us	187 us
P99	176 us	105 us
Exit code	1 (queue cleanup ENOENT)	0

Distribution (histogram counts)

Wake-up latency	SCHED_OTHER	SCHED_FIFO
0-100 us	826 (82.6%)	985 (98.5%)
100-500 us	174 (17.4%)	15 (1.5%)
500-1000 us	0	0
1000 us and over	0	0

P99 was lower by 40.34% and average latency was lower by about 78.11% for SCHED_FIFO in these runs. This is a measured association, not proof that the policy alone caused the entire difference: additional diagnostics were run during the SCHED_OTHER measurement, causing possible CPU and message-queue interference. Its exit code 1 corresponded to mq_unlink ENOENT at shutdown; the 1,000 acquired samples remain present. Both tests ran on the Host, not on ARM.

```
ptx-0002's current scheduling priority: 0
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$ cd ~/embedded_asgn2
echo "===== SCREENSHOTS ====="
find screenshots -maxdepth 1 -type f \
  \( -iname '*.png' -o -iname '*.jpg' -o -iname '*.jpeg' \) \
  -printf '%f\n' | sort

echo "===== TOTAL SCREENSHOTS ====="
find screenshots -maxdepth 1 -type f \
  \( -iname '*.png' -o -iname '*.jpg' -o -iname '*.jpeg' \) \
  | wc -l
===== SCREENSHOTS =====
===== TOTAL SCREENSHOTS =====
0
nguyendangquang@nguyendangquang-Bellator:~/embedded_asgn2$
```

Figure 13. Original diagnostic collection context; definitive 1,000-sample measurements and counts are transcribed from the completed Terminal comparison of 10 October 2026.

21. GDB and perf: Follow-up Diagnostics

GDB alert-branch verification (Host x86_64)

GDB breakpoint: `check_sensor_alerts()` in `processor_thread.c:115`, called from `processor_thread_main()` at line 208. GDB info threads identified the main thread plus four workers; the backtrace reached `start_thread` and `clone3`. Normal Host input 42.5 C / 85% triggered HIGH TEMPERATURE (threshold 40 C) and HIGH HUMIDITY (threshold 80%).

For spike branch testing, GDB changed `previous_temp` from 42.5 C to 25.0 C while `has_previous=true` and current temperature remained 42.5 C. Measured difference became 17.5 C. Application output confirmed TEMPERATURE SPIKE | Value: 17.5 | Threshold: 5.0. This was an intentional debugger state injection and is not a run driven by two distinct CSV temperature readings.

CPU profiling (Host x86_64)

Test	Observed value
perf stat, 30.10 s wall time	14.95 ms task-clock, 476 context switches, 36 migrations, 178 faults
Approximate CPU utilization	0.05% task-clock / elapsed-time ratio
perf record -e cpu-clock -F 999 -p PID	8 CPU-clock samples attributed to <code>sms_app_host</code>
One observed hotspot	<code>__kmalloc_node_noprof</code> in kernel, 12.5% (1/8 samples)
Another observed hotspot	<code>__schedule</code> in kernel, 12.5% (1/8 samples)

The allocation call chain was `processor_thread_main -> check_sensor_alerts -> send_alert -> mq_timedsend -> do_mq_timedsend -> load_msg -> __kmalloc_node_noprof`. The scheduling chain included `nanosleep -> __schedule`. These are real samples from SMS; 8 samples is too few to rank stable bottlenecks or claim a statistically reliable hotspot. Optimization ideas: avoid unnecessary repeated alerts, preserve blocking waits instead of busy loops, and consider a longer representative workload.

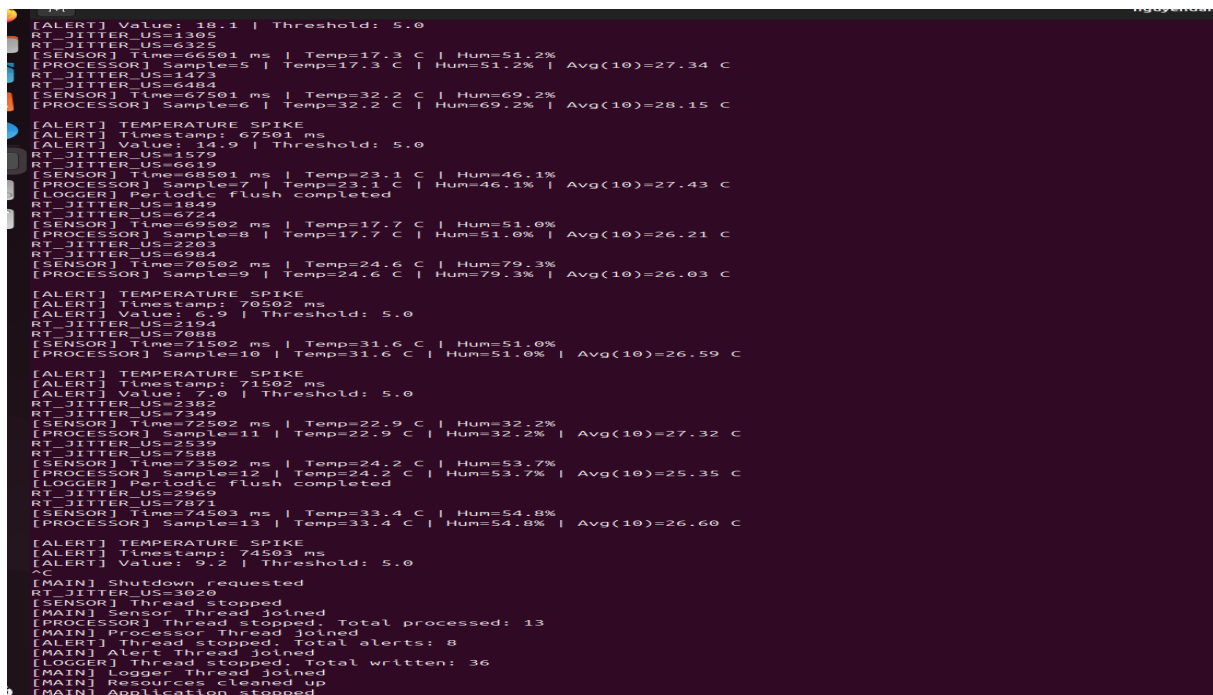
22. ARM QEMU Real-Time Verification and Final Status

An independent 32-bit statically linked ARM EABI5 executable, `sms_app_arm_rt`, was built from the existing ARM code while preserving driver ioctl behavior. A separate `initramfs_arm_rt.cpio.gz` was created; the original image and application remained unchanged. This version ran in `vexpress-a9` QEMU using Linux 5.15 and the SMS kernel module.

Observed ARM console evidence

```
[RT] mlockall enabled
[RT] Requested=SCHED_FIFO Actual=SCHED_FIFO Priority=50
RT_JITTER_US=...
[MAIN] Application stopped
```

The actual scheduling policy and priority were read by `pthread_getschedparam()` inside Sensor Thread. A short observed ARM run processed 5 sensor samples, handled 4 alerts, wrote 16 log records and shut down normally after SIGINT. Additional ARM runs also showed direct wake-up latencies such as 1,305 us, 6,325 us, 1,473 us and 6,484 us; those isolated values are not a 1,000-sample ARM distribution or an ARM P99.



```
[ALERT] Value: 18.1 | Threshold: 5.0
RT_JITTER_US=1305
RT_JITTER_US=6325
[SENSOR] Time=66501 ms | Temp=17.3 C | Hum=51.2%
[PROCESSOR] Sample=5 | Temp=17.3 C | Hum=51.2% | Avg(10)=27.34 C
RT_JITTER_US=1473
RT_JITTER_US=6484
[SENSOR] Time=67501 ms | Temp=32.2 C | Hum=69.2%
[PROCESSOR] Sample=6 | Temp=32.2 C | Hum=69.2% | Avg(10)=28.15 C

[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 67501 ms
[ALERT] Value: 34.9 | Threshold: 5.0
RT_JITTER_US=1579
RT_JITTER_US=6619
[SENSOR] Time=68501 ms | Temp=23.1 C | Hum=46.1%
[PROCESSOR] Sample=7 | Temp=23.1 C | Hum=46.1% | Avg(10)=27.43 C
[LOGGER] Periodic flush completed
RT_JITTER_US=1849
RT_JITTER_US=6724
[SENSOR] Time=69502 ms | Temp=17.7 C | Hum=51.0%
[PROCESSOR] Sample=8 | Temp=17.7 C | Hum=51.0% | Avg(10)=26.21 C
RT_JITTER_US=2203
RT_JITTER_US=6984
[SENSOR] Time=70502 ms | Temp=24.6 C | Hum=79.3%
[PROCESSOR] Sample=9 | Temp=24.6 C | Hum=79.3% | Avg(10)=26.03 C

[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 70502 ms
[ALERT] Value: 6.9 | Threshold: 5.0
RT_JITTER_US=2194
RT_JITTER_US=7088
[SENSOR] Time=71502 ms | Temp=31.6 C | Hum=51.0%
[PROCESSOR] Sample=10 | Temp=31.6 C | Hum=51.0% | Avg(10)=26.59 C

[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 71502 ms
[ALERT] Value: 7.0 | Threshold: 5.0
RT_JITTER_US=2382
RT_JITTER_US=7349
[SENSOR] Time=72502 ms | Temp=22.9 C | Hum=32.2%
[PROCESSOR] Sample=11 | Temp=22.9 C | Hum=32.2% | Avg(10)=27.32 C
RT_JITTER_US=539
RT_JITTER_US=7588
[SENSOR] Time=73502 ms | Temp=24.2 C | Hum=53.7%
[PROCESSOR] Sample=12 | Temp=24.2 C | Hum=53.7% | Avg(10)=25.35 C
[LOGGER] Periodic flush completed
RT_JITTER_US=2909
RT_JITTER_US=7871
[SENSOR] Time=74503 ms | Temp=33.4 C | Hum=54.8%
[PROCESSOR] Sample=13 | Temp=33.4 C | Hum=54.8% | Avg(10)=26.60 C

[ALERT] TEMPERATURE SPIKE
[ALERT] Timestamp: 74503 ms
[ALERT] Value: 9.2 | Threshold: 5.0
[MAIN] Shutdown requested
RT_JITTER_US=3020
[SENSOR] Thread stopped
[MAIN] Sensor Thread joined
[PROCESSOR] Thread stopped. Total processed: 13
[MAIN] Processor Thread joined
[ALERT] Thread stopped. Total alerts: 8
[MAIN] Alert Thread joined
[LOGGER] Thread stopped. Total written: 36
[MAIN] Logger Thread joined
[MAIN] Resources cleaned up
[MAIN] Application stopped
```

Figure 14. Follow-up ARM QEMU terminal: sensor data, wake-up latency lines, temperature-spike alerts and orderly four-thread shutdown.

Outstanding requirements / cautions

The 1,000-per-mode jitter comparison was performed on the Host, while the successful FIFO 50 / mlockall proof is from ARM. The GDB injection and perf capture also occurred on the Host. Remaining or not-shown requirements: ARM-scale jitter statistics if requested on target; `strace -c` summary ranked by elapsed time; exact mandatory GDB scenario wording; `-i/-t/-h/-d/-v` CLI functionality; rolling humidity average; and a separate 3-5-minute video. This technical report does not claim those items as completed.